

Announcements

- Lectures today and next Monday (9/14) will be given by me
- Paper presentations start next Wednesday (9/16)
 - [Paper summary requirements](#)
- [Sign up for teams](#) by next Monday (9/14)

All the Non-system Stuff You Need to Know About:

CSE 585 Advanced Scalable Systems for Agentic AI

September 8, 2026

Kevin Xue

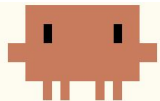
Adapted from materials originally created by
Shiqi He and Jae-Won Chung

Kevin Xue

- 3rd-year PhD student advised by Mosharaf and Ryan Huang
 - CSE 585 by Mosharaf: systems for AI
 - CSE 598 by Ryan: AI for systems (consider taking that as well!)
- Research
 - Multiplexing hardware for on-device GenAI inference
 - Disaggregated inference
- Office hours: Fridays 11:30 am - 12:30 pm @ BBB 4828

Agents?

Writing Software with Claude Code



Claude Code v2.1.246
Opus 5 (1M context) · Claude Pro
~/snake

Auto mode is now Claude Code's default permission mode.

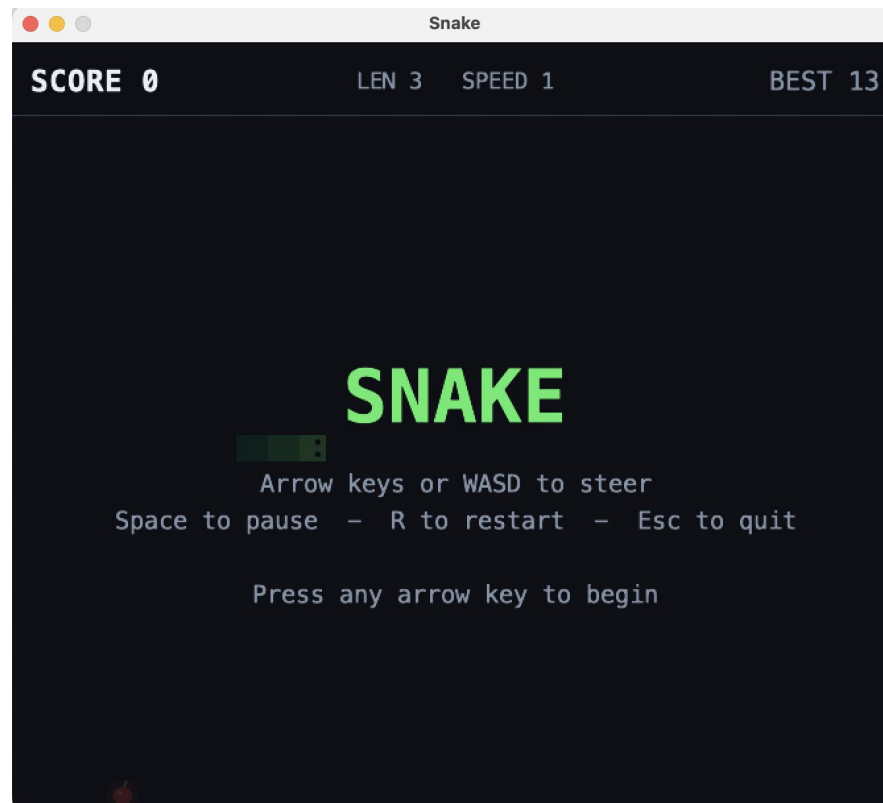
Auto mode lets Claude handle permission prompts automatically. Claude checks each tool call for risky actions and prompt injection before executing, runs the ones it assesses as lower-risk, and blocks the rest.
<https://code.claude.com/docs/en/permission-modes>
+3 more · /status

› Write in ~/snake a classic Snake Game with the Arcade library. You should refer to its documents for usage of API. <https://docs.arcade.dev/en/home>.

Figure out all dependencies by yourself. You should use uv to install a virtual python environment.

The snake game should be able to played with ~/snake/play.

Writing Software with Claude Code



Claude Code = Harness + Model



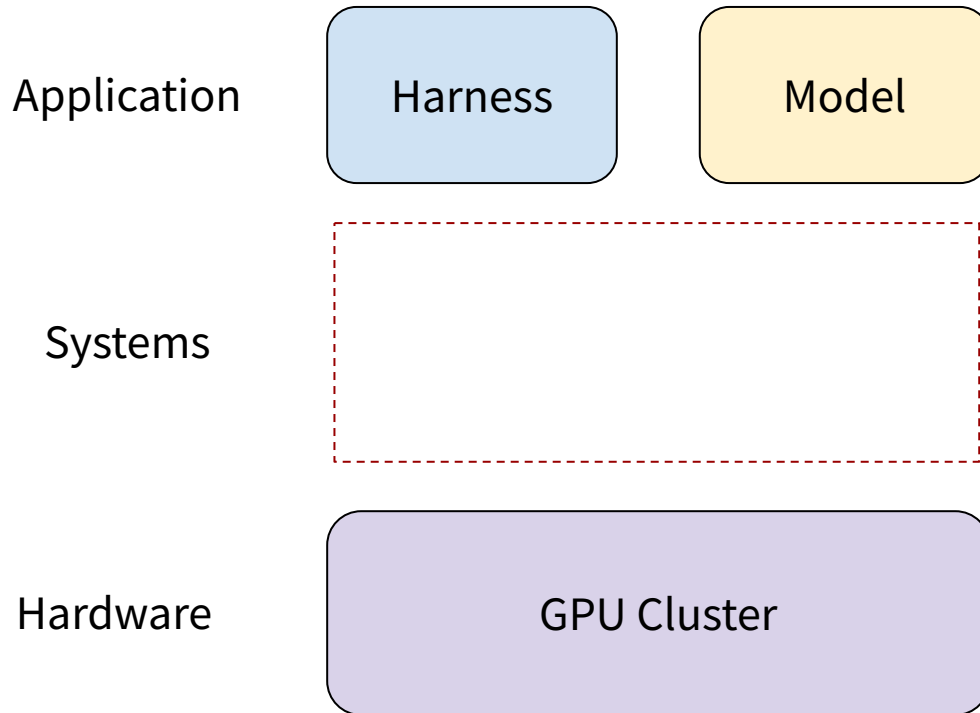
Model or Harness?

1. Evaluates my prompt and believes it should read the doc I provided online
2. Claude Code uses Fetch tool to read it, and sends the result to Opus 5
3. Finds out that the doc's URL I provided is wrong!
4. Figures out the correct URL and determines the correct packages to install
5. Installs the packages in a virtual environment
6. Comes up with the first version of the code
7. Writes the first version into a set of files

Model or Harness?

- [M] Evaluates my prompt and believes it should read the doc I provided online
- [H] Claude Code uses Fetch tool to read it, and sends the result to Opus 5
- [M] Finds out that the doc's URL I provided is wrong!
- [M] Figures out the correct URL and determines the correct packages to install
- [H] Installs the packages in a virtual environment
- [M] Comes up with the first version of the code
- [H] Writes the first version into a set of files

Systems?



Today: Understanding our Application

Model

- Path to the ChatGPT moment
- **Systems for models!**

Harness

- Walking through a loop
- **Systems for harness!**

Systems for agents!

Application

Harness

Model

Systems

Hardware

GPU Cluster

Models

Why Care about Models

* Churned for 5m 26s done 1:19 AM

[Settings](#) [Status](#) [Config](#) [Usage](#) [Stats](#)

Session

Total cost: \$1.55
Total duration (API): 4m 39s
Total duration (wall): 11m 5s
Total code changes: 0 lines added, 0 lines removed
Usage by model:
 claude-haiku-4-5: 22.6k input, 328 output, 0 cache read, 0 cache write (\$0.0243)
 claude-opus-5: 546 input, 22.8k output, 1.0m cache read, 44.9k cache write (\$1.52)

Lots of time spent in model inference
Inference is running on \$\$\$\$\$

~82 tokens per second

Path to the ChatGPT Moment

- **NLP and transformers**
- Generative pre-training
- Scaling law
- Post-training alignment

Natural Language Processing

Long time ago, scientists wanted computers to do:

- **Translation:** What is the Japanese of “That’s impossible”?
 - Answer: ありえない (“a-ri-ei-na-ee”)
- **Sentiment analysis:** Did the reviewer like the movie?
 - “Best film ever. It cured my insomnia!”
- **Document Q&A:** Is Nvidia stock a buy based on its Form 10-K in 2016?
 - “No. AMD’s new GPUs are more friendly for gamers. Nvidia is losing the competition.”

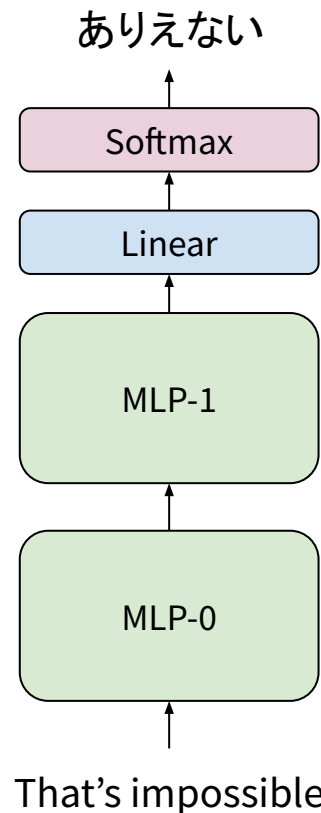
Deep Learning 101

Core component: multi-layer perceptron (MLP):

- A linear transformation layer
- A non-linear transformation layer (e.g., ReLU)
- Repeat multiple times

Just MLP for language modeling: cannot understand the **relationship between words** in sentences

(History of RNN & LSTM is skipped)



Attention is All You Need (Google, 2017)

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Łukasz Kaiser*
Google Brain
lukaszkaier@google.com

Illia Polosukhin* ‡
illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.0 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature.

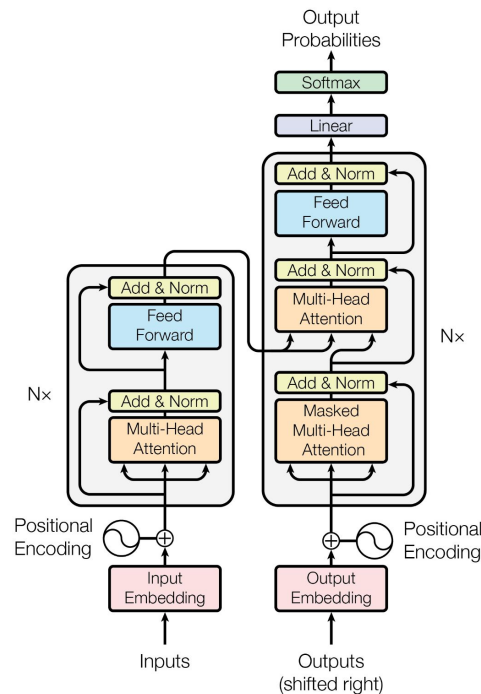
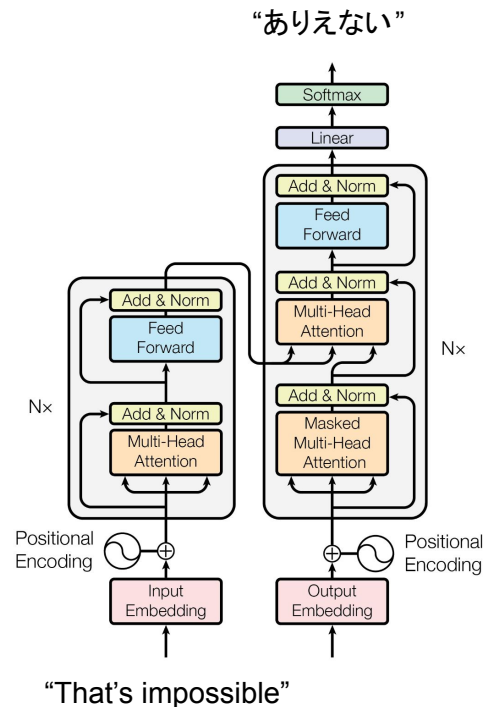
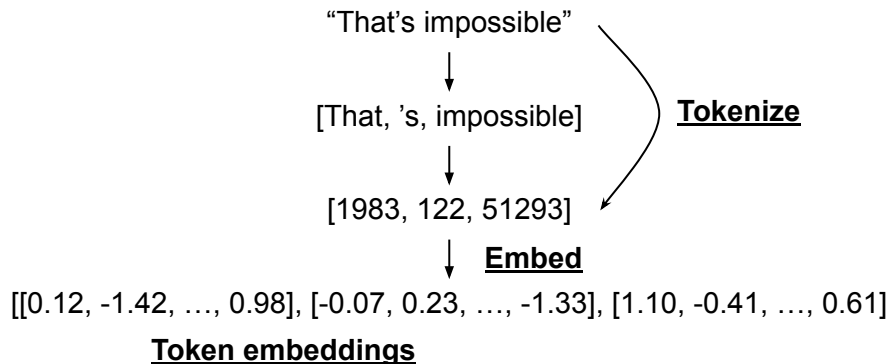


Figure 1: The Transformer - model architecture.

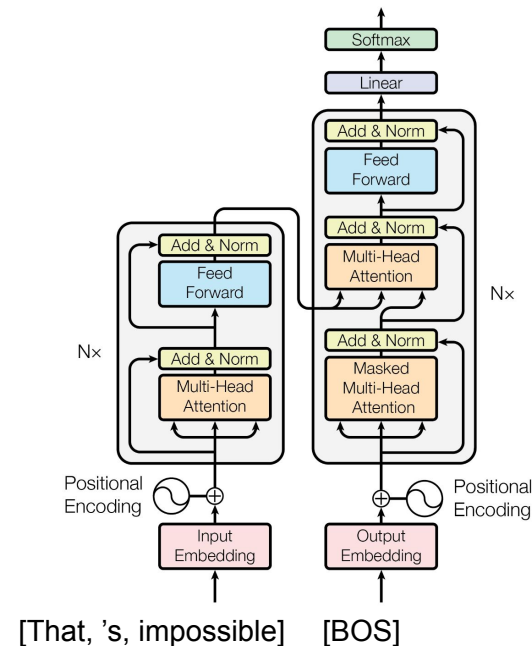
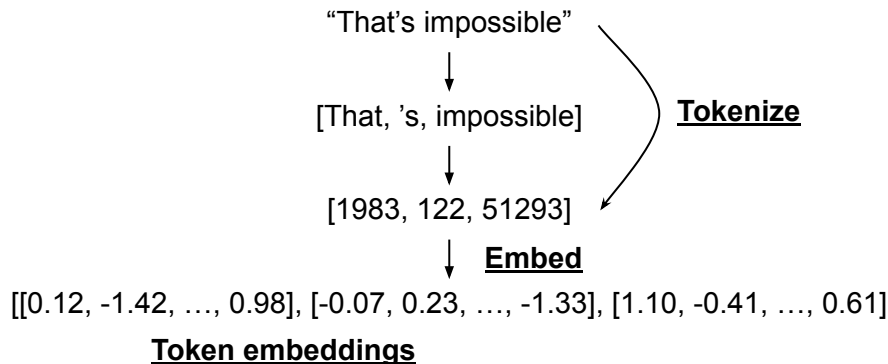
Attention is All You Need (Google, 2017)

- The Transformer model
- Critical Components
 - Tokenization & embedding
 - Attention
 - MLP



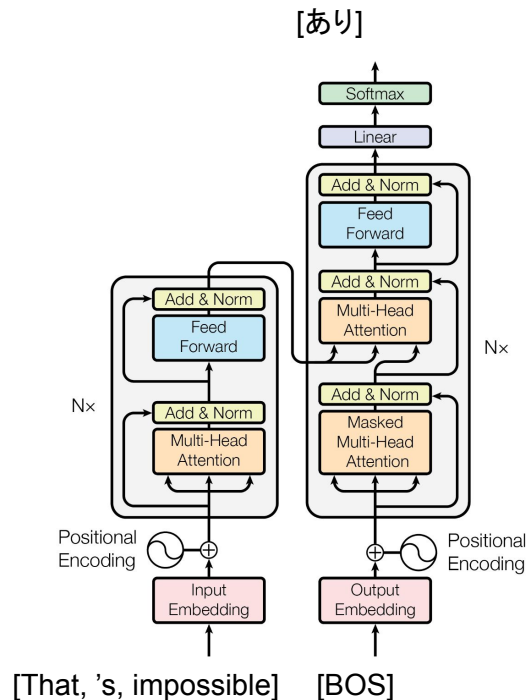
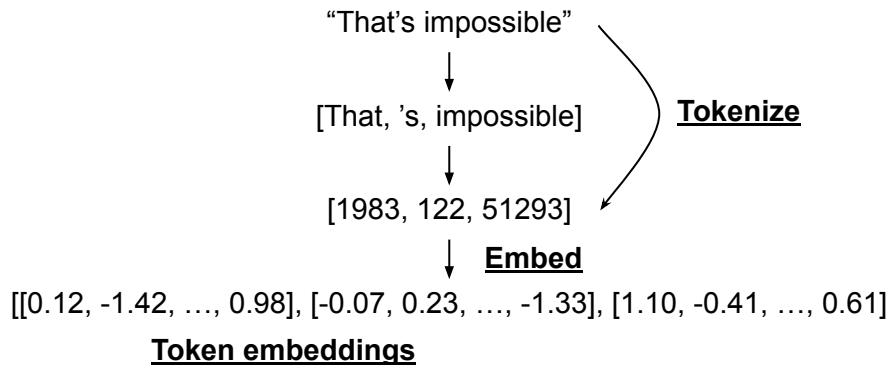
Attention is All You Need (Google, 2017)

- Autoregressive text generation
 - One forward pass produces one output token
 - Keep appending output token to input
 - Stop when the special EOS token is produced



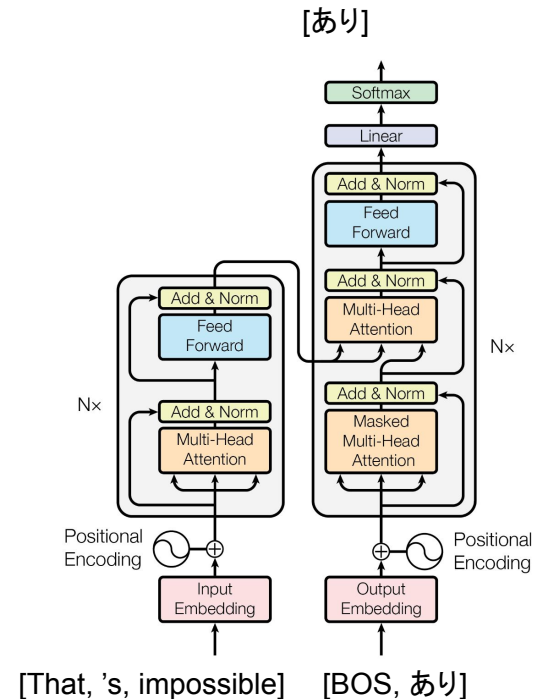
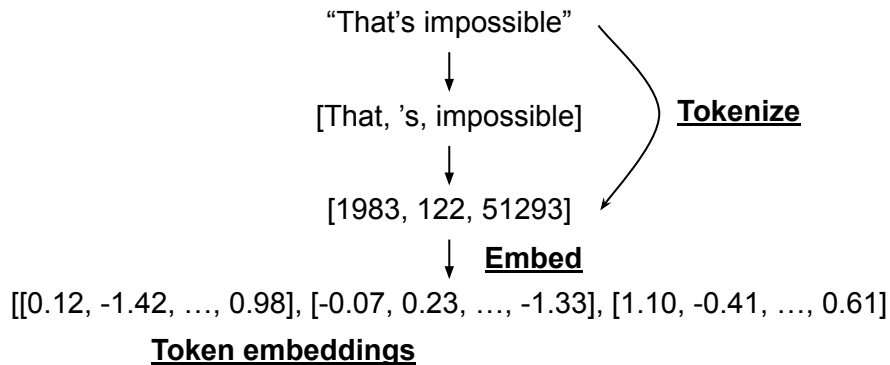
Attention is All You Need (Google, 2017)

- Autoregressive text generation
 - One forward pass produces one output token
 - Keep appending output token to input
 - Stop when the special EOS token is produced



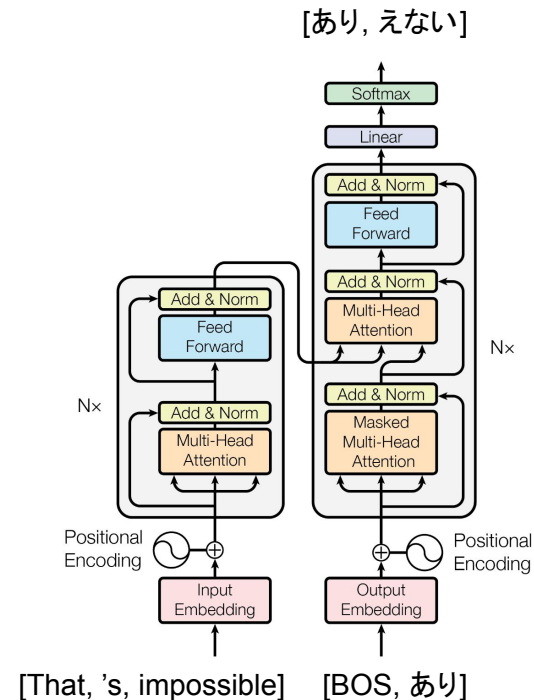
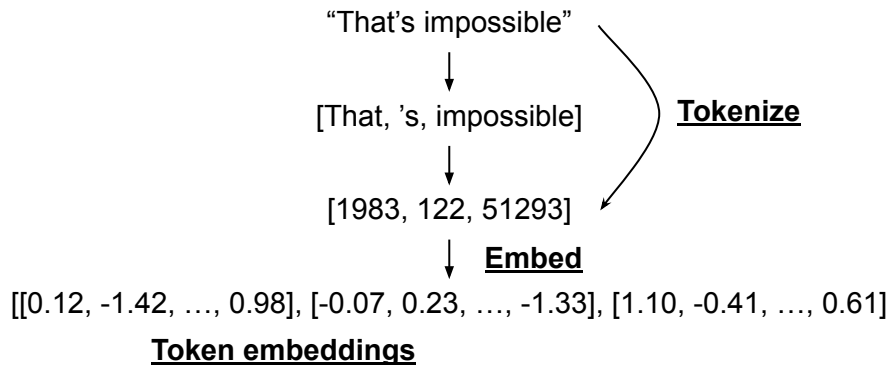
Attention is All You Need (Google, 2017)

- Autoregressive text generation
 - One forward pass produces one output token
 - Keep appending output token to input
 - Stop when the special EOS token is produced



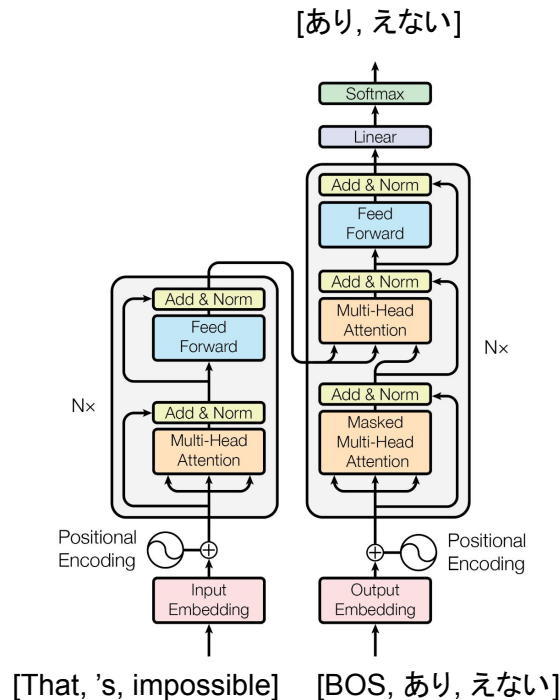
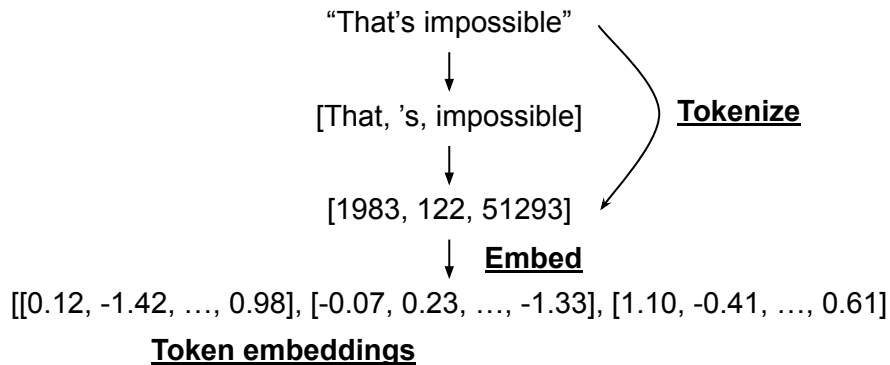
Attention is All You Need (Google, 2017)

- Autoregressive text generation
 - One forward pass produces one output token
 - Keep appending output token to input
 - Stop when the special EOS token is produced



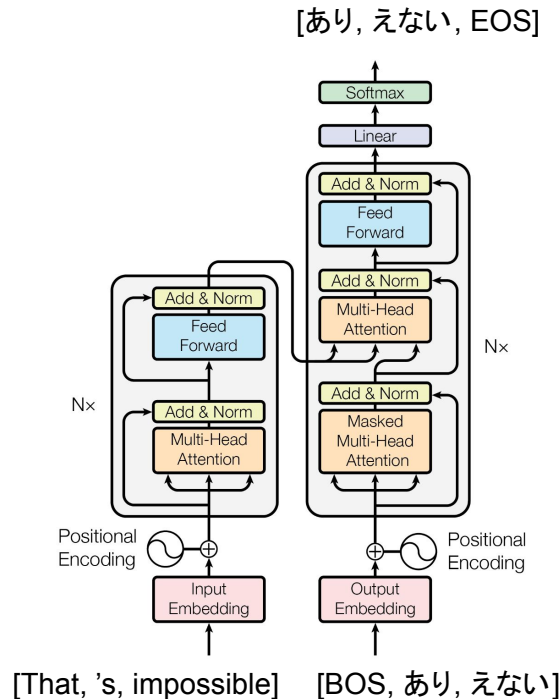
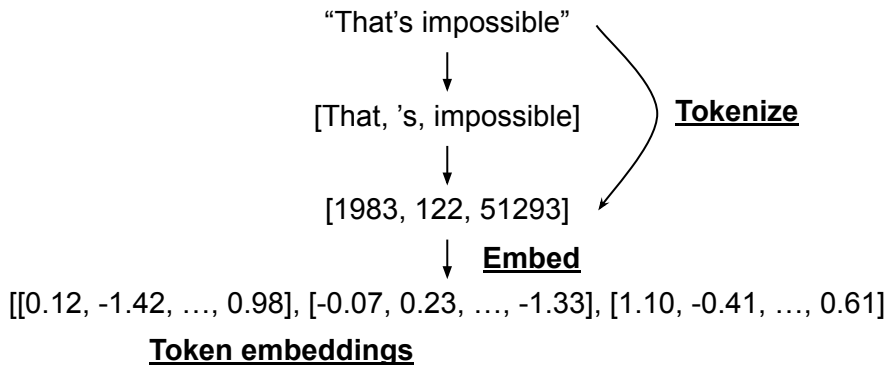
Attention is All You Need (Google, 2017)

- Autoregressive text generation
 - One forward pass produces one output token
 - Keep appending output token to input
 - Stop when the special EOS token is produced



Attention is All You Need (Google, 2017)

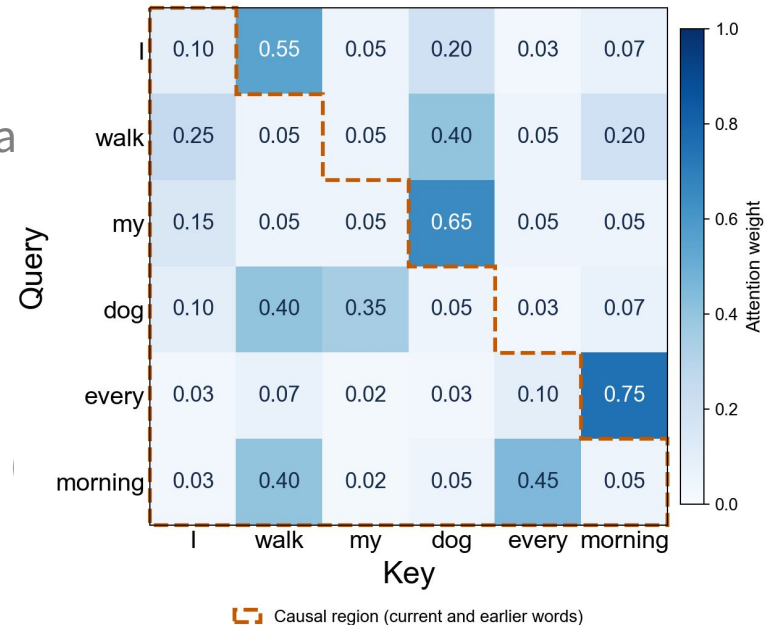
- Autoregressive text generation
 - One forward pass produces one output token
 - Keep appending output token to input
 - Stop when the special EOS token is produced



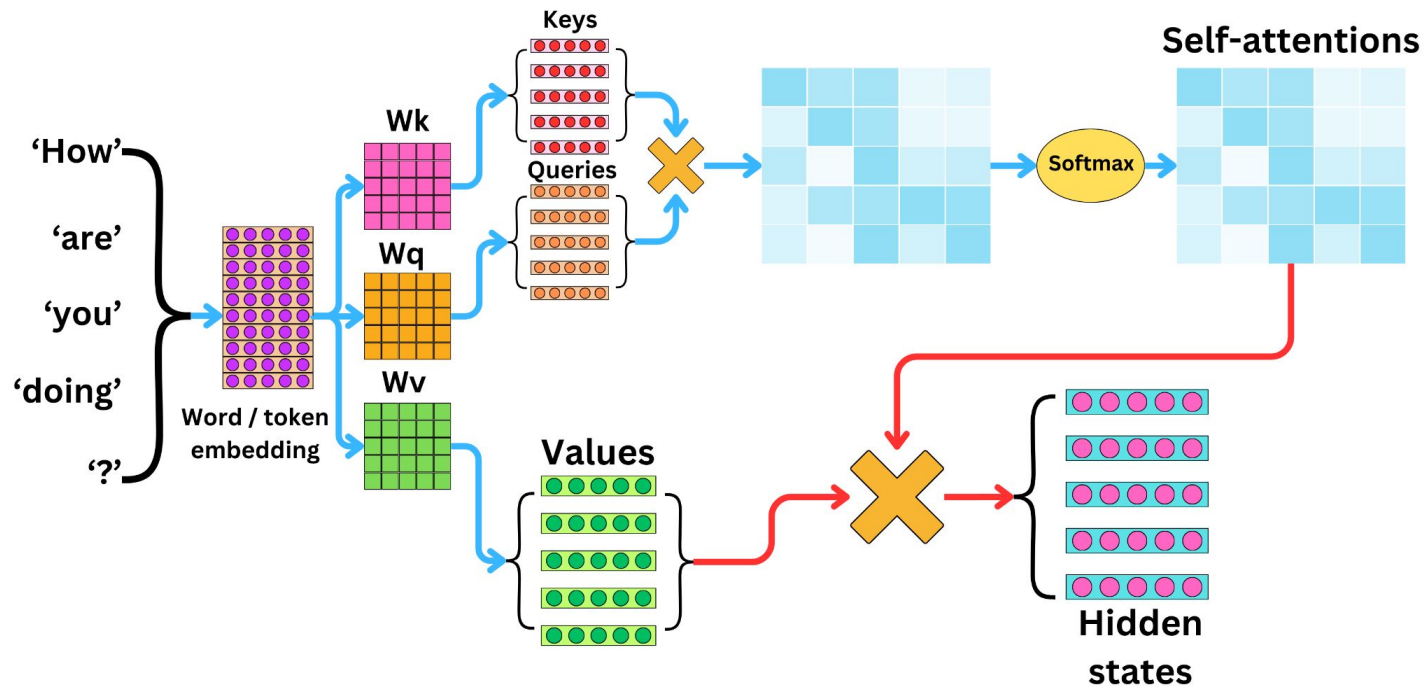
Self-attention

Encodes **relationships between tokens**

- Goal: add this information to the hidden state process
- Encoder: relationship in both directions
- Decoder: relationship in causal region only



Self-attention

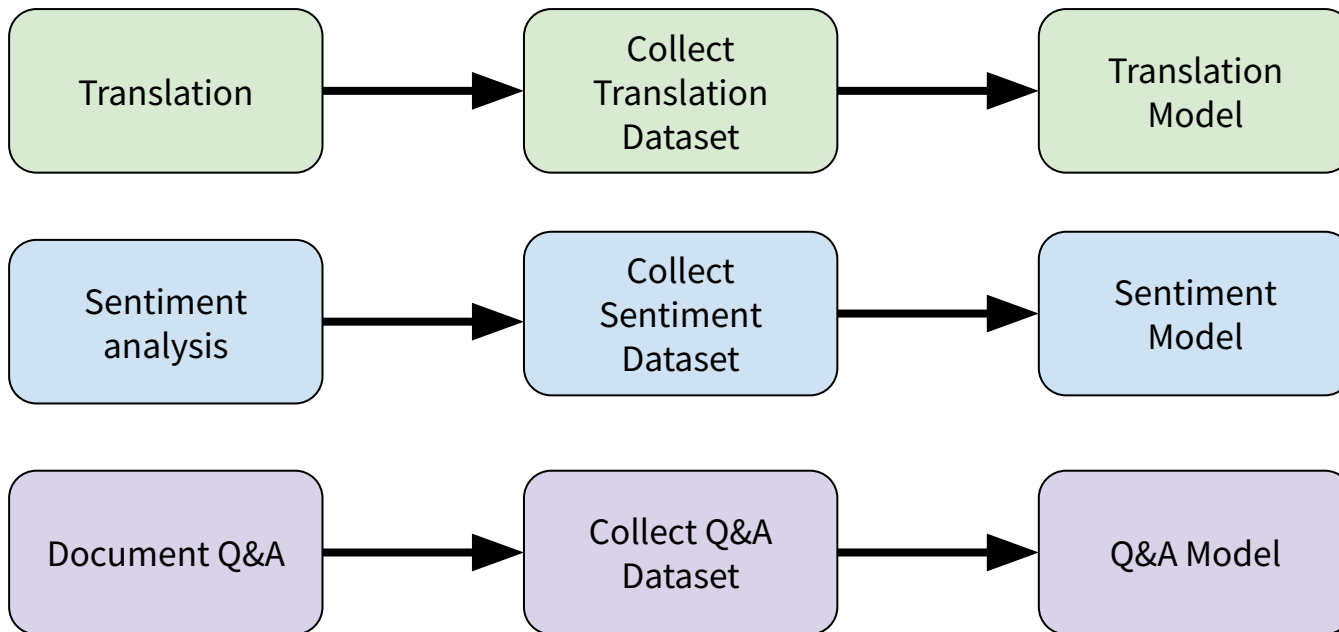


<https://newsletter.theaiedge.io/p/understanding-the-self-attention>

Path to the ChatGPT Moment

- NLP and transformers
- **Generative pre-training**
- **Scaling law**
- **Post-training alignment**

Traditional NLP Solutions



You need to find a large labeled dataset for each task!

Causal Language Modeling

Could all those tasks be next token prediction tasks?

Given a sequence of tokens $X = (x_1, x_2, \dots, x_{t-1})$, predict the next token x_t

- (“Japanese”, “for”, “that’s impossible”, is) → “ありえない”
- Goal: make the predicted sequence closer to the real-world distribution
 - Real world data does not need labeling!

$$P(x) = \prod_{t=1}^T P(x_t \mid x_1, x_2, \dots, x_{t-1})$$

GPT (OpenAI, 2018)

- “Improving Language Understanding by Generative Pre-Training”
- A new paradigm
 - All tasks are next token prediction tasks
 - First, pre-train a large model on a **large unlabeled** corpus of text
 - Then, fine-tune the model on a **small labeled** task-specific dataset (e.g., translation)
 - Now, you can pre-train a large model once and reuse it for many downstream tasks

GPT-1 (OpenAI, 2018)

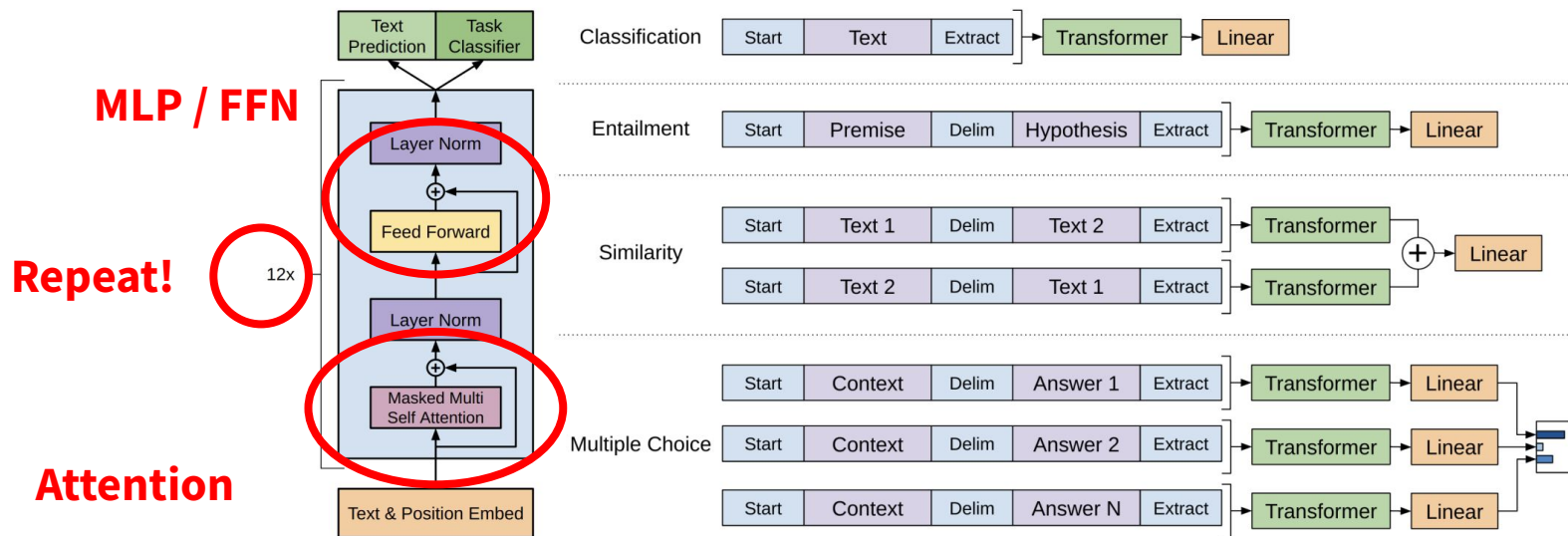


Figure 1: **(left)** Transformer architecture and training objectives used in this work. **(right)** Input transformations for fine-tuning on different tasks. We convert all structured inputs into token sequences to be processed by our pre-trained model, followed by a linear+softmax layer.

GPT-2 (OpenAI, 2019)

- Successor to GPT
 - [Better language models and their implications](#) (OpenAI blog)
 - 1.5B parameters (12.8x), 40GB of Internet text (8x)

System Prompt
(human-written)

In a shocking finding, scientist discovered a herd of unicorns living in a remote, previously unexplored valley, in the Andes Mountains. Even more surprising to the researchers was the fact that the unicorns spoke perfect English.

Model Completion
(machine-written,
10 tries)

The scientist named the population, after their distinctive horn, Ovid's Unicorn. These four-horned, silver-white unicorns were previously unknown to science.

Now, after almost two centuries, the mystery of what sparked this odd phenomenon is finally solved.

Dr. Jorge Pérez, an evolutionary biologist from the University of La Paz, and several companions, were exploring the Andes Mountains when they found a small valley, with no other animals or humans. Pérez noticed that the valley had what appeared to be a natural fountain, surrounded by two peaks of rock and silver snow.

Scaling Laws (OpenAI, Kaplan et al., Jan 2020)

- GPT-1 to GPT-2
 - Model parameters: 117M $\rightarrow$ 1.5B (12.8x)
 - Data: 5 GB $\rightarrow$ 40 GB (8x)
- Questions
 - Should we continue scaling the number of parameters and the dataset?
 - What's the best ratio between scaling model parameters and the dataset?
 - Equivalently, given the same amount of target compute, what's the (#params, data size) pair that leads to the best model quality?

Scaling Laws (OpenAI, Kaplan et al., Jan 2020)

- Questions
 - Should we continue scaling the number of parameters and the dataset?
 - What's the best ratio between scaling model parameters and the dataset?
- “Let’s do a grid search and see what happens.”
 - [Scaling Laws for Neural Language Models](#) (OpenAI)

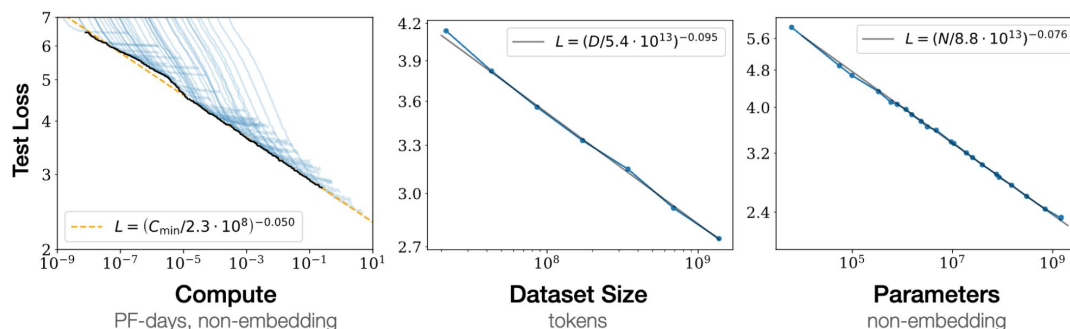


Figure 1 Language modeling performance improves smoothly as we increase the model size, dataset size, and amount of compute² used for training. For optimal performance all three factors must be scaled up in tandem. Empirical performance has a power-law relationship with each individual factor when not bottlenecked by the other two.

Scaling Laws (OpenAI, Kaplan et al., Jan 2020)

- Questions *and* Kaplan et al.'s answer
 - Should we continue scaling the number of parameters and the dataset? *Yeah, probably.*
 - What's the best ratio between scaling model parameters and the dataset? *Roughly equally.*
- “Let’s do a grid search and see what happens.”
 - [Scaling Laws for Neural Language Models](#) (OpenAI)

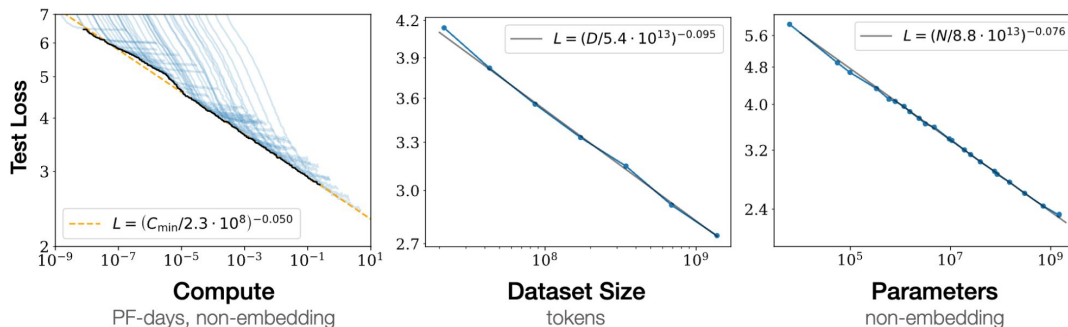


Figure 1 Language modeling performance improves smoothly as we increase the model size, dataset size, and amount of compute² used for training. For optimal performance all three factors must be scaled up in tandem. Empirical performance has a power-law relationship with each individual factor when not bottlenecked by the other two.

Scaling Laws (OpenAI, Jan 2020)

- Two notable progressions after this
 - [Chinchilla scaling law](#) (Google DeepMind, Hoffman et al., 2022)
 - You should actually scale data more!

Scaling Laws (OpenAI, Jan 2020)

- Two notable progressions after this
 - [Chinchilla scaling law](#) (Google DeepMind, Hoffman et al., 2022)
 - You should actually scale data more!
 - Overtraining: Train models on more data than the Chinchilla-optimal point
 - Your training time compute is no longer *optimal* (in Chinchilla sense)
 - You end up with a smaller model with good model quality
 - But smaller models are cheaper to run inference on!

LLaMA: Open and Efficient Foundation Language Models

Hugo Touvron*, Thibaut Lavril*, Gautier Izacard*, Xavier Martinet
Marie-Anne Lachaux, Timothee Lacroix, Baptiste Rozière, Naman Goyal
Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin
Edouard Grave*, Guillaume Lample*

Meta AI

The objective of the scaling laws from [Hoffmann et al. \(2022\)](#) is to determine how to best scale the dataset and model sizes for a particular *training* compute budget. However, **this objective disregards the inference budget**, which becomes critical when serving a language model at scale. In this context, given a target level of performance, the preferred model is not the fastest to train but the fastest at inference, and although it may be cheaper to train a large model to reach a certain level of performance, a smaller one trained longer will ultimately be **cheaper at inference**. For instance, although [Hoffmann et al. \(2022\)](#) recommends training a 10B model on 200B tokens, we find that the performance of a 7B model continues to improve even after 1T tokens.

GPT-3 (OpenAI, May 2020)

- GPT-3
 - [Language Models are Few-Shot Learners](#) (OpenAI paper)
 - 175B parameters, 400B tokens
- A new paradigm
 - Previously: Pre-train on large corpus, and then fine-tune on task-specific dataset.
 - GPT-3: Don't even do the fine-tuning part. Instead, put a **few** examples in the prompt.

Few-shot examples

Actual prompt |
GPT's completion |

```
Poor English input:  I eated the purple berries.
Good English output: I ate the purple berries.
Poor English input:  Thank you for picking me as your designer.  I'd appreciate it.
Good English output: Thank you for choosing me as your designer.  I appreciate it.
Poor English input:  The mentioned changes have done.  or I did the alteration that you
requested.  or I changed things you wanted and did the modifications.
Good English output: The requested changes have been made.  or I made the alteration that you
requested.  or I changed things you wanted and made the modifications.
Poor English input:  I'd be more than happy to work with you in another project.
Good English output: I'd be more than happy to work with you on another project.
```

RLHF (OpenAI, Jan 2022)

- Reinforcement Learning with Human Feedback (RLHF)
 - [Aligning language models to follow instructions](#) (OpenAI blog)
- Few-shot prompting works, but it has limitations
 - Hope: We want GPT to do text completion in a super friendly way
 - But nothing in the pre-training corpus really endorses the model to be friendly
 - We do need a step after pre-training to reinforce certain behaviors

RLHF (OpenAI, Jan 2022)

Three steps

- Among many GPT responses to the same prompt, a human indicates which one they prefer
- Train a *reward model* that predicts human preference for a given response
- Run RL on the LLM using the reward model (**PPO**; Proximal Policy Optimization)

Path to the ChatGPT Moment

- NLP and transformers
- Generative pre-training
- Scaling law
- Post-training alignment

Introducing ChatGPT

We've trained a model called ChatGPT which interacts in a conversational way. The dialogue format makes it possible for ChatGPT to answer followup questions, admit its mistakes, challenge incorrect premises, and reject inappropriate requests.

ChatGPT is a sibling model to InstructGPT, which is trained to follow an instruction in a prompt and provide a detailed response.

We are excited to introduce ChatGPT to get users' feedback and learn about its strengths and weaknesses. During the research preview, usage of ChatGPT is free. Try it now at chatgpt.com.

ChatGPT (OpenAI, Nov 2022)

- Still a text generation model that predicts the next token!
- But the prompt is formatted in a particular way

Real prompt rendering of the OpenAI GPT OSS 20B model.
The **template** is baked into the model during **post-training**.

System prompt: You are a helpful assistant.
User's prompt: Write a poem about systems.

**Prompt
rendering**

```
<|start|>system<|message|>You are ChatGPT, a  
large language model trained by OpenAI.  
Knowledge cutoff: 2024-06  
Current date: 2025-08-28  
Reasoning: medium  
  
# Valid channels: analysis, commentary, final.  
Channel must be included for every  
message.<|end|><|start|>developer<|message|>#  
Instructions  
  
You are a helpful assistant.  
  
<|end|><|start|>user<|message|>Write a poem  
about systems.<|end|><|start|>assistant
```

ChatGPT (OpenAI, Nov 2022)

- Still a text generation model that predicts the next token!
- But the prompt is formatted in a particular way

Real prompt rendering of the OpenAI GPT OSS 20B model.
The **template** is baked into the model during **post-training**.

System prompt: You are a helpful assistant.
User's prompt: Write a poem about systems.

**Prompt
rendering**

```
<|start|>system<|message|>You are ChatGPT, a  
large language model trained by OpenAI.  
Knowledge cutoff: 2024-06  
Current date: 2025-08-28  
Reasoning: medium  
  
# Valid channels: analysis, commentary, final.  
Channel must be included for every  
message.<|end|><|start|>developer<|message|>#  
Instructions  
  
You are a helpful assistant.  
  
<|end|><|start|>user<|message|>Write a poem  
about systems.<|end|><|start|>assistant
```

Pre-training Large Language Models

You are tasked with training Claude's Orchestra 6 model

- The model size is 5 Trillion (5 TB in FP8)
- 100 TB of data is used to train the model
- You have a few Vera Rubin NVL 72 racks (each rack = 288 GB GPUs * 72)

What problems could arise?

How to Partition the Model on GPUs?

- Put different layers on different GPUs?
 - They are sequential, would some GPUs be idle?
- Split each layer and put them on different GPUs?
 - They are sending gradient updates like crazy! All the time is spent on communication

Other Pre-training Problems

- Much compute and memory requirements
- Much GPU
 - Like, 100k or more for a single training run
 - We are already in the era of multi-datacenter training
 - How are we going to split one model training into 100k+ GPUs?
- Much communication
 - The 100k GPUs need to talk to each other
 - If not optimized well, expensive GPUs are going to spend all their lives doing communication
- Much power and energy
 - An H100 GPU draws anywhere between 100 to 700 W during training, and we have 100k of it
 - Building power plants take years but we need that power now
- Much failure
 - Say a GPU fails at any moment with probability 0.01%
 - The probability of no GPUs failing is $(1 - 0.0001)^{100000} = 0.00004538$

LLM Inference

After you have ChatGPT, you need to deploy it somewhere such that people can use it.

- How to partition the model (again)?
- How to generate texts for multiple prompts in parallel?
- How to squeeze your GPU's behavior such that it is doing useful work?
- How to manage the priorities of requests?

Managing the KV Cache

- LLM inference stores and reuses the key and value matrices to prevent repeating insane amount of compute when generating each token
 - More on this during the inference lecture
- KV cache grows linearly with context length!
 - Worse, you don't know how long they will be

Parallel Processing of Prompts

Problem:

- Your SLO is the first token must be generated within 1 second
- Each prompt takes 5 seconds to finish
- But you have only 8 GPUs

Solution: batching requests (adding dimension to matrix multiplication)

- But prompts start from different time!
- What if the KV cache doesn't fit the memory any more?

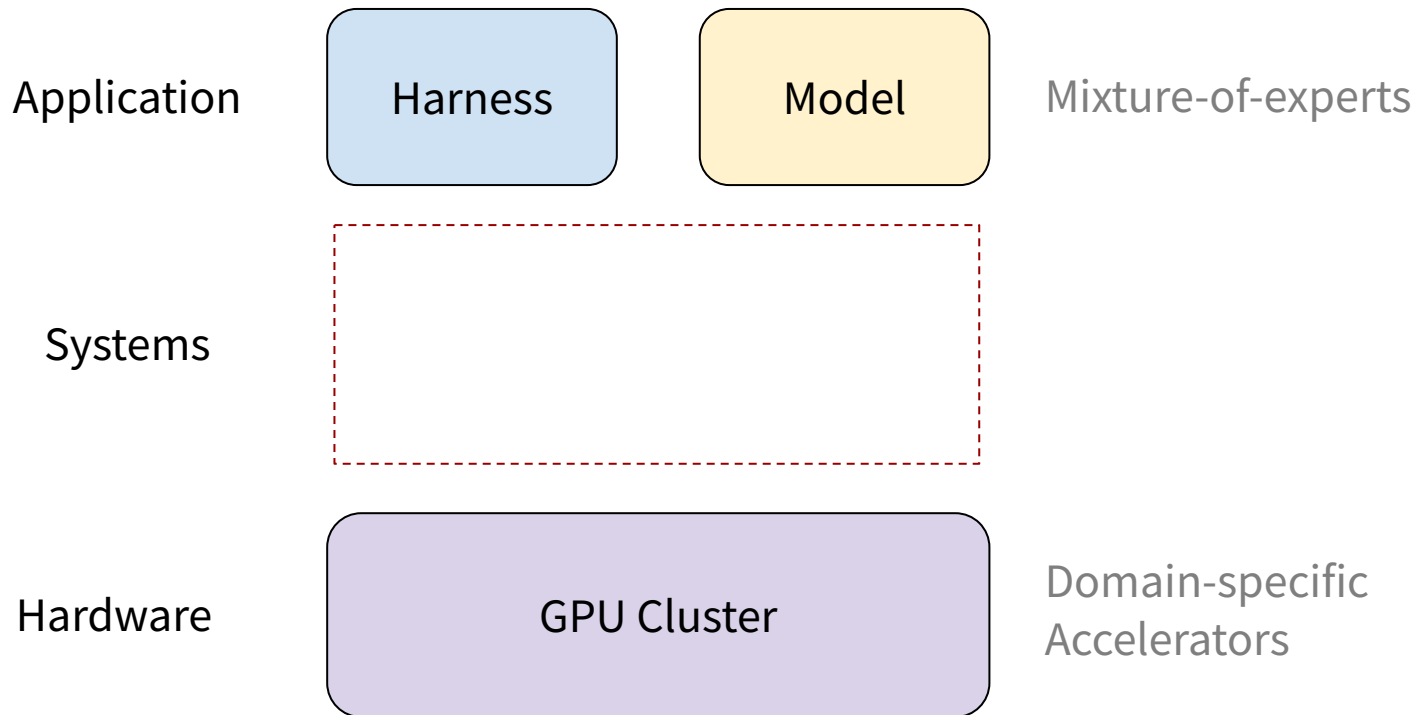
Resource Usage

Problem:

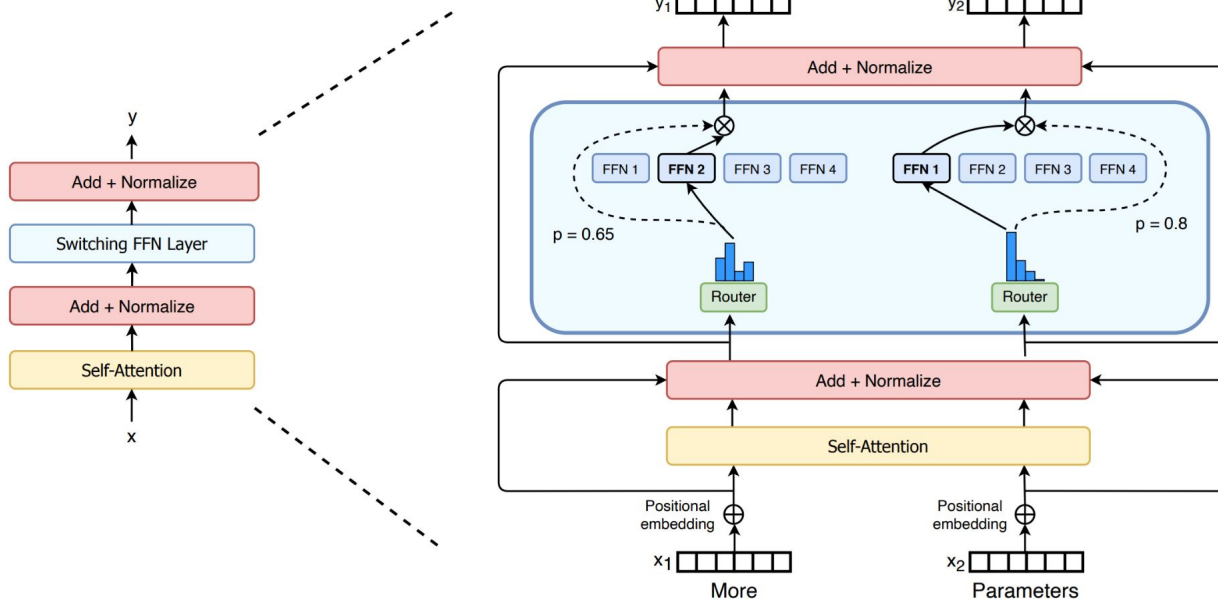
- Inference has two phases: the prompt processing phase and the text generation phase
- Prompt processing phase is compute-bound (all tokens are processed in parallel)
- Text generation phase is memory-bound (processing only one token)
- Compute is idle when doing prompt processing!

Solution: find some strong GPUs for prompt processing and weaker ones for generation

Non-system Solutions to System Problems



Example: Mixture-of-Experts



Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity

MoE: A Model Solution to System Problem

Computes only a small subset of FFN layers

- Saves compute and energy
- Saves memory bandwidth usage (because fewer memory is streamed to chips)

But also causes more problems:

- How to partition experts on different GPUs? Load balancing?
- When serving, what if all requests are using different experts?

That's why there are always jobs in systems (yet).

Beyond Standard Transformers

(We have 1 lecture covering those)

- Multi-modal models
- Mamba and alternative attention algorithms
- Diffusion models

Harness & Agents

From LLM to Agents

“Find the cheapest nonstop flight from DTW to SFO next Friday and tell me whether it conflicts with my calendar.”

A vanilla LLM does not know your calendar or recent flight prices

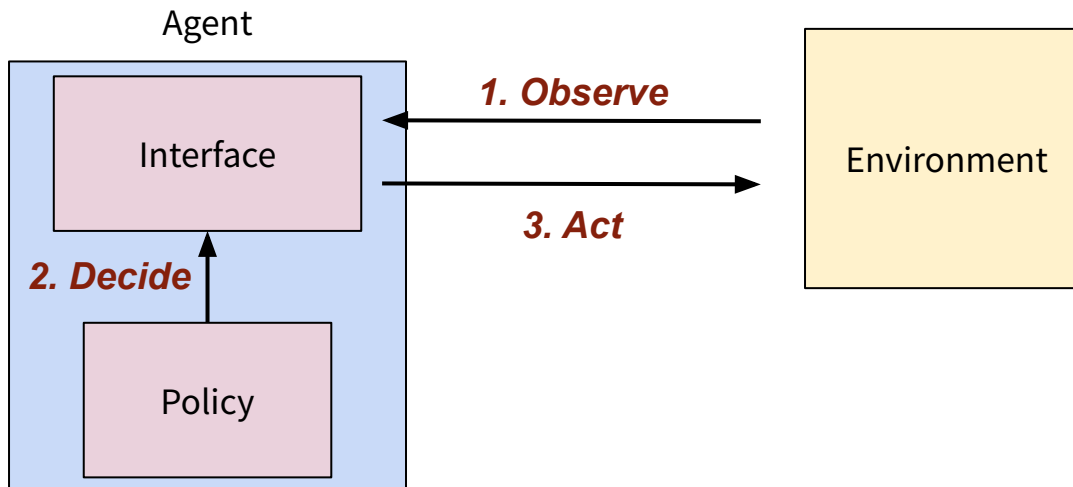
Idea: human use tools to solve this problem. Create tools for LLM as well:

- `search_flights()`
- `get_calendar()`

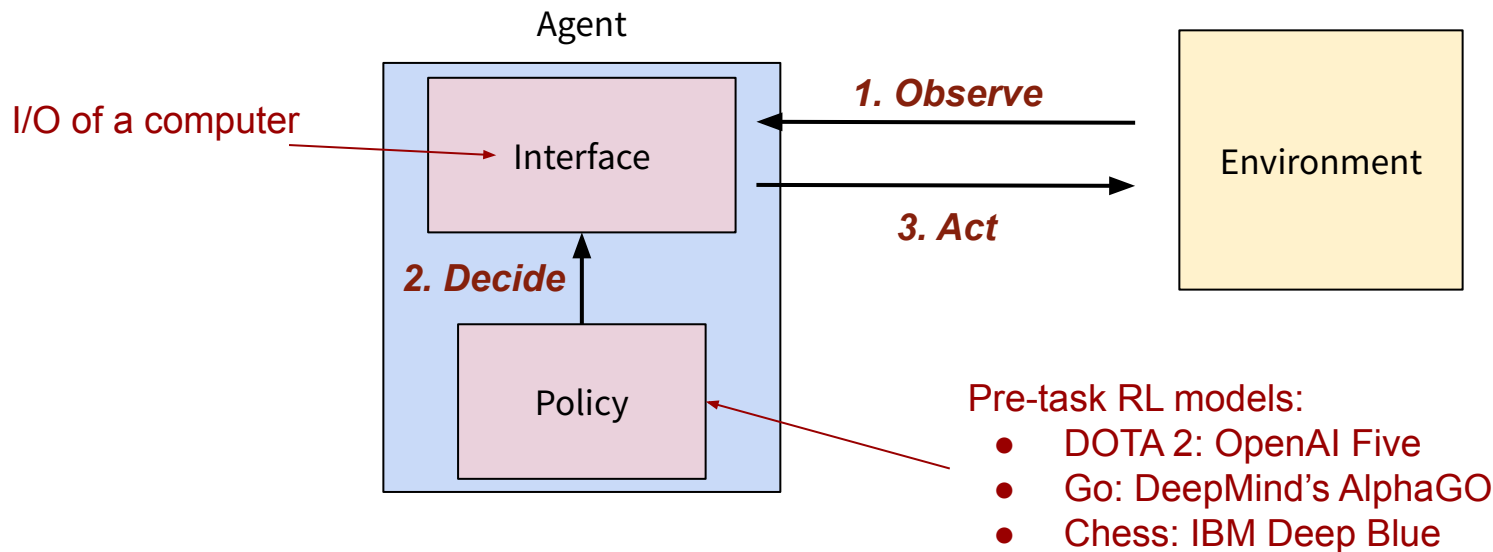
Definition of an AI Agent

Russell and Norvig 1995:

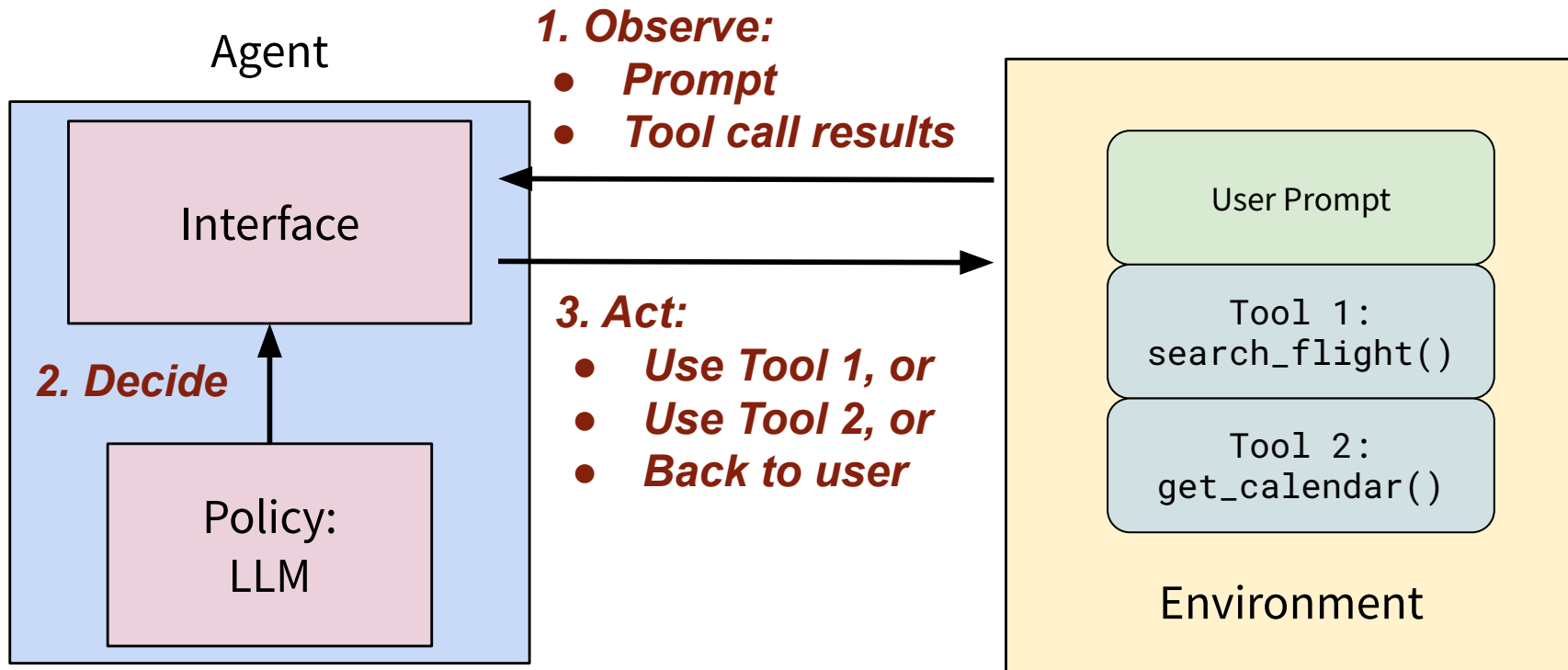
“Anything that can be viewed as perceiving its environment through sensors and acting upon that environment through actuators”



Traditional Agents: RL / Rule-based Policies



LLM Agents: Policy = LLM



A Minimal Agentic Loop

```
messages = [user_request]

while True:
    response = llm(messages, tools)

    if response.tool_call:
        result = execute(response.tool_call)
        messages += [response, result]
    else:
        return response.text
```

Notes:

- Tools are registered to the LLM
- LLM responses using a set format for the tool call
- Loop will only end when response is no longer a tool call

First LLM Request: Prompt

System:

You are a travel assistant.

Available tools:

- **search_flights(origin, destination, date)**

User:

Find me a flight from DTW to SFO on September 11.

First LLM Request: Response

```
<tool_call>

{
  "name": "search_flights",
  "arguments": {
    "origin": "DTW",
    "destination": "SFO",
    "date": "2026-09-11"
  }
}

</tool_call>
```

Harness: Processing LLM's Response

1. LLM returns with a tool call rather than a text response
2. Don't get back to user yet
3. Parse the arguments for the tool call
4. Execute the **search_flights()** call with the parsed argument
5. Parse the response to the tool call, and append it to the context with LLM
6. Send another request to the LLM with the new context with the tool call response

Second LLM Request Prompt

Prompt

User:

Find me a flight from DTW to SFO on September 11.

Assistant:

<tool_call>

```
search_flights(origin="DTW", destination="SFO",  
               date="2026-09-11")
```

</tool_call>

Tool:

```
[  
  {"flight": "DL123", "departure": "08:30", "price": 329},  
  {"flight": "UA456", "departure": "14:10", "price": 287}  
]
```

Output:

UA456 is the cheaper option at \$287, departing DTW at 2:10 PM.

(No additional tool calls, can return to the user)

Blindly Using Tools is Not Helpful

(1) Hotspot QA

Question: Aside from the Apple Remote, what other device can control the program Apple Remote was originally designed to interact with?

(1a) Standard

Answer: iPod



(1b) CoT (Reason Only)

Thought: Let's think step by step. Apple Remote was originally designed to interact with Apple TV. Apple TV can be controlled by iPhone, iPad, and iPod Touch. So the answer is iPhone, iPad, and iPod Touch.

Answer: iPhone, iPad, iPod Touch



(1c) Act-Only

Act 1: Search[Apple Remote]
Obs 1: The Apple Remote is a remote control ...

Act 2: Search[Front Row]
Obs 2: Could not find [Front Row]. Similar: ...

Act 3: Search[Front Row (software)]
Obs 3: Front Row is a discontinued media center software ...

Act 4: Finish[yes]



(1d) ReAct (Reason + Act)

Thought 1: I need to search Apple Remote and find the program it was originally designed to interact with.

Act 1: Search[Apple Remote]

Obs 1: The Apple Remote is a remote control introduced in October 2005 by Apple ... originally designed to control the Front Row media center program ...

Thought 2: Apple Remote was originally designed to control the Front Row media center program. I need to search Front Row next and find what other device can control it.

Act 2: Search[Front Row]

Obs 2: Could not find [Front Row]. Similar: ['Front Row Seat to Earth', 'Front Row Motorsports', 'Front Row (software)', ...]

Thought 3: Front Row is not found. I need to search Front Row (software) .

Act 3: Search[Front Row (software)]

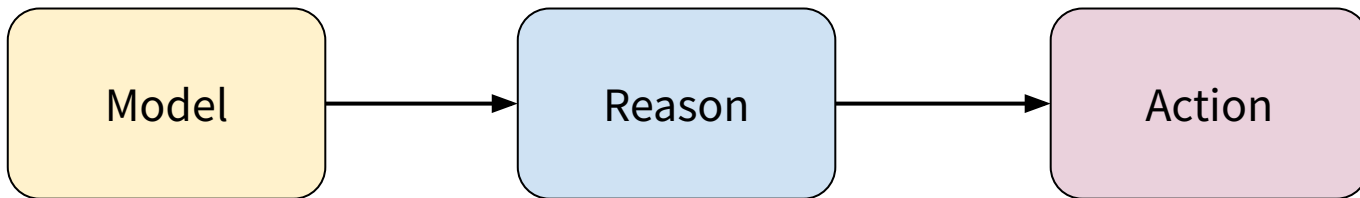
Obs 3: Front Row is a discontinued media center software ...

Thought 4: Front Row (software) is controlled by an Apple Remote or the keyboard function keys. So the answer is keyboard function keys.

Act 4: Finish[keyboard function keys]



ReAct (Google Brain, 2023): Think Before Acting

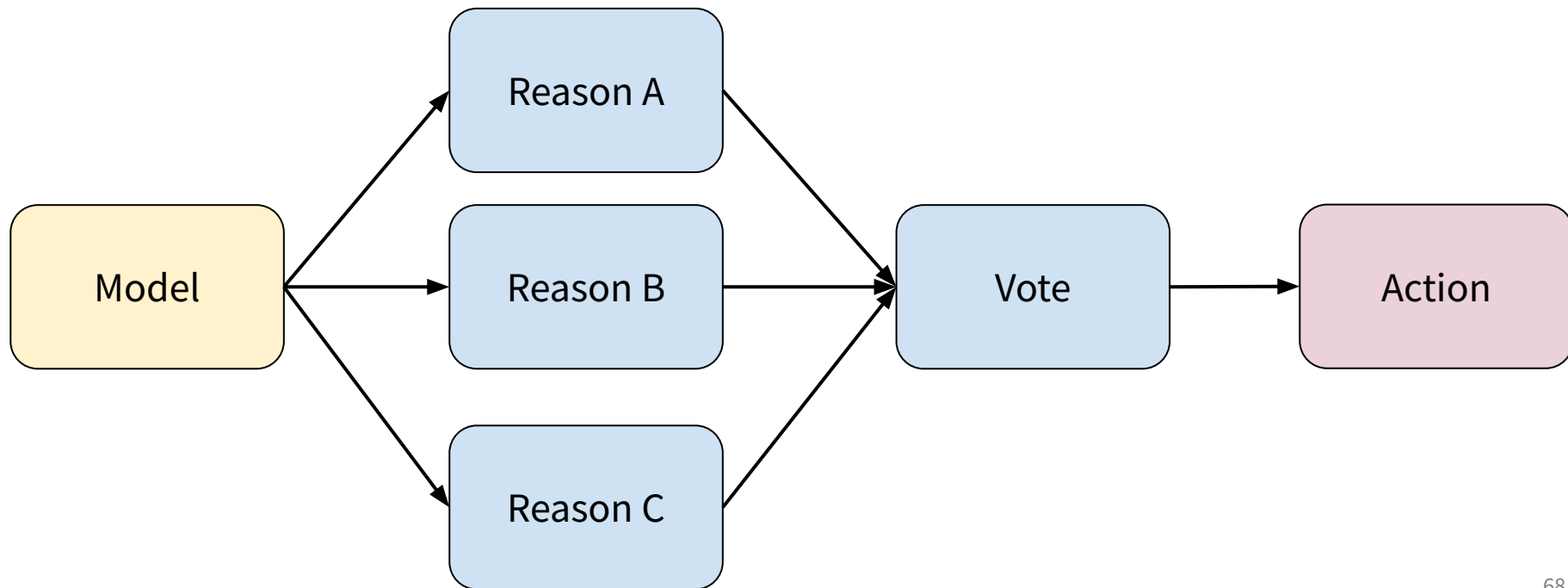


Reasoning: “answering questions that require complex, multi-step generation with intermediate steps”

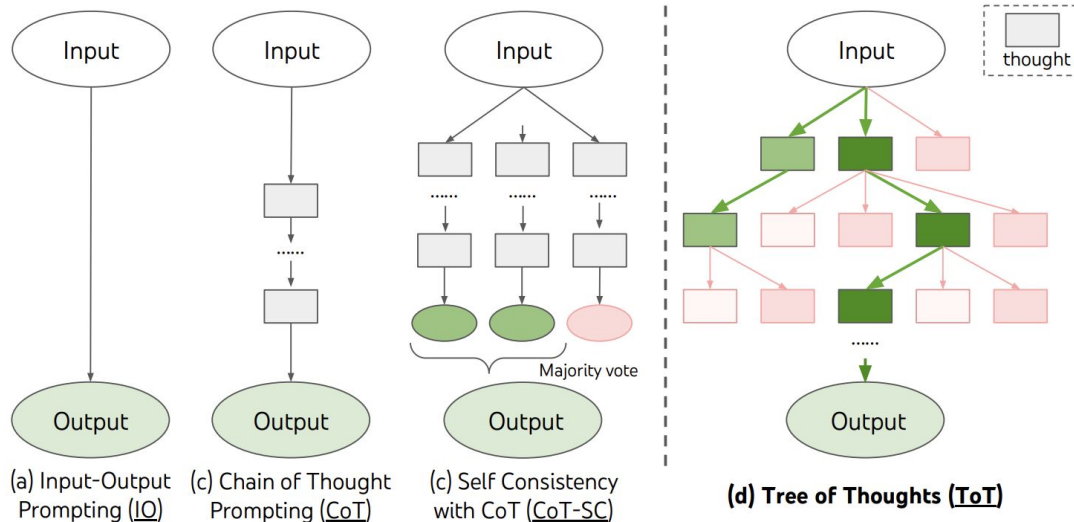
(From [Understanding Reasoning LLMs](#))

Self-Consistency (Google Brain, 2023)

Reasons multiple different ways before acting (i.e., prompting LLMs in parallel)



Tree-of-thoughts (Princeton & DeepMind 2023)



[Yao et al., NeurIPS 2023](#)

Other Harness Components

- **Structured plans:** breaking down complicated tasks into smaller, more manageable parts
- **Memory:** distinguish short-term and long-term memory to track progress
- **Access to additional tools:** e.g., MCP, Agent2Agent
- **Permission:** which tools are allowed in a certain environment
- **Subagent:** ask another agent to complete a subtask

More in today's required [Claude Code Deep Dive](#) Reading

Systems for Agentic AI

“Wow, it’s only a loop, and you created half a course for it??”

GIVE ME 5 MINUTES TO CONVINCE YOU!

“LLM Serving is ~~Compute-Bound~~”

From NanoFlow (OSDI 2025)

- True, **if batch size can be 512**
- With the number of tokens that will be generated by tool calls and CoT, this is often less than 16

Memory-bound means huge waste of compute!

ric that determines serving systems’ performance. Due to large model sizes and memory-intensive self-attention, LLM serving has been commonly assumed to be memory-bound. Through a detailed analysis, we show that despite having memory-intensive components, end-to-end LLM serving is **compute bound** for most common workloads and LLMs. Alas, most existing serving engines fall short from optimal compute utilization, because the heterogeneous operations that comprise LLM serving—compute, memory, networking—are executed sequentially within a device.

Beyond Individual Text Completion

One agentic task contains many inference API requests

- Co-scheduling requests belonging to the same user is important
- These are different graphs, requiring graph optimization techniques

Some requests may spend lots of time executing tool calls

- How about their KV cache?
 - Evict? Do you know for sure they are long tool calls?

Systems for Agents to Use

Traditional way of thinking about systems:

- New workload comes out, build systems for them to run more efficiently

What about: **an agent instead of a human is using the system, how are we supposed to build it?**

- Your Codex might start many Git Worktrees, how to checkpoint them efficiently?
- Some software stack is built for human readability that sacrifices performance, can we rethink those stacks?
- Would the permission systems be different for agents than humans?

Systems Research in Agentic AI

- Coding becomes cheap
 - You can do lots of exploration way more efficiently than before
 - But we might lose understanding of details that can give you research ideas
- Passing-level paper/doc writing becomes easier
 - Great news for non-native speakers
 - But good and honest communication still need to be made from human
- **We encourage you to discuss how you use agents in your projects. Share it on Ed with your peers, teach us as well!**